

TD7: Ingeniería de Datos

Adecuación a Big Data

Estado de situación

Generación de las Batch Views

— — —



Hasta acá la **Batch Layer** cuenta con dos capacidades:

- ❖ **Almacenamiento distribuido** para albergar el **Master Dataset**
 - Pensado para muy **grandes volúmenes** de datos
 - Con alto **throughput** (gracias a la distribución y replicación)
 - Garantizando **high availability** (gracias a la replicación)
 - **Escalable horizontalmente** considerando el constante crecimiento del dataset
- ❖ **Cómputo distribuido** para poder procesar ese dataset

Generación de las Batch Views



— — —

¿Pero qué tipo de información va a poder generarse con estas capacidades? ¿Y cómo se almacena para ser consumido?

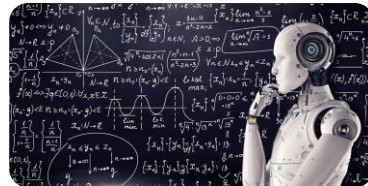
Algunos ejemplos para pensar:

- ❖ Modelo de **Machine Learning**
- ❖ **Índices invertidos**
- ❖ **Collaborative filtering**

Modelos de Machine Learning

Clasificador bayesiano

— — —



Modelo de Machine Learning:

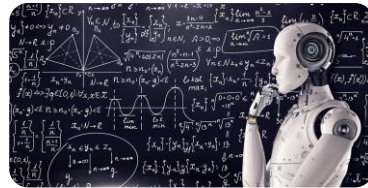
- ❖ Un **algoritmo o representación matemática** que se entrena con datos para reconocer patrones, predecir resultados o tomar decisiones

Un caso básico de modelo de ML es el **clasificador bayesiano**.

En nuestro ejemplo, un **clasificador binario** para predecir si un mensaje es o no spam.

Clasificador bayesiano

— — —

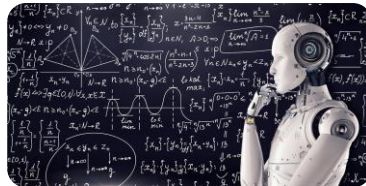


Entrenar un modelo va a implicar un corpus de entrenamiento.

En este caso va a surgir desde el **Master Dataset**:

- ❖ Nuestros **facts** serán eventos en los que un usuario marcó un mensaje como **spam** o como no spam (**ham**).
- ❖ Este **dataset** crece en el tiempo, agrandando el corpus y mejorando el modelo.
- ❖ Mientras tanto, nuestros **algoritmos** de entrenamiento pueden **evolucionar**.

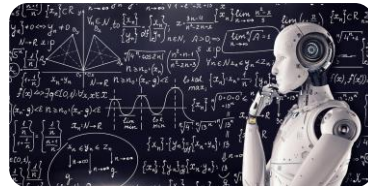
Clasificador de spam



$$\Pr(S|W) = \frac{\Pr(W|S) \cdot \Pr(S)}{\Pr(W|S) \cdot \Pr(S) + \Pr(W|H) \cdot \Pr(H)}$$

- ❖ **$\Pr(S|W)$** es la probabilidad de que un mensaje sea spam si contiene la palabra W
- ❖ **$\Pr(W|S)$** es la probabilidad de que la palabra W aparezca en los mensajes marcados como spam en nuestro corpus
- ❖ **$\Pr(S)$** es la probabilidad de que un mensaje al azar sea spam
- ❖ **$\Pr(W|H)$** es la probabilidad de que la palabra W aparezca en los mensajes marcados como no spam (ham)
- ❖ **$\Pr(H)$** es la probabilidad de que un mensaje al azar no sea spam

Clasificador de spam - MapReduce

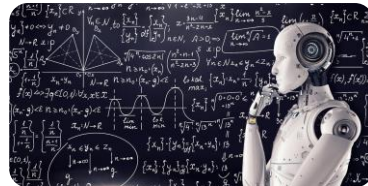


— — —

Cómo podemos resolver esto con MapReduce:

```
1  def mapper(key, value):
2      # key: label (spam o no spam)
3      # value: el mensaje clasificado
4      words = value.tokenize()
5
6      # Se emite un 1 para el total de la label
7      yield (f"TOTAL_LABEL_{key}", 1)
8
9      for word in words:
10         # Se emite un 1 para cada combinación label/word
11         yield (f"COUNT_{key}_{word}", 1)
12         # Se emite un 1 para la palabra
13         yield (f"TOTAL_WORD_{word}", 1)
```

Clasificador de spam - MapReduce

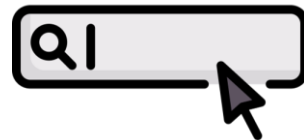


```
1 def reducer(key, values):
2     # key: un total o el count para un par word/label
3     # values: la lista de counts del mapper
4     count = sum(values)
5
6     yield (key, count)
```

- ❖ Teniendo los **totales de mensajes** marcados como spam y ham en el corpus y la cantidad de veces que aparece cada palabra en cada label se pueden calcular las **probabilidades**
- ❖ Sólo se necesita una forma de accederlas eficientemente

Índices invertidos

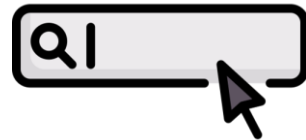
Índices invertidos



— — —
¿Qué es un **índice invertido**?

- ❖ Estructura (índice) que permite buscar por parte o partes de un elemento más grande
- ❖ Son la base de los **buscadores de texto**
- ❖ Pueden incluir información para brindar:
 - Búsquedas por frase
 - Búsquedas por proximidad
 - Ordenamiento por relevancia textual

Índices invertidos

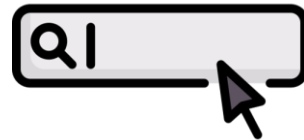


— — —

En tanto estructura:

- ❖ Almacenan **tokens** (palabras) comunes a muchos elementos (documentos)
- ❖ Permiten encontrar un token con **alta performance** ($O(1)$ u $O(\log(n))$)
- ❖ Permiten encontrar tokens a partir de un **prefijo**
- ❖ Por cada token, guardan una **colección de elementos** en los que ese token aparece
 - Adicionalmente pueden almacenar la **posición** (o posiciones) en la que aparece dentro del elemento

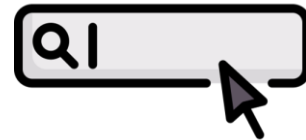
Índices invertidos - MapReduce



```
1 def mapper(key, value):
2     # key: identificador del documento
3     # value: contenido del documento
4     words = value.tokenize()
5
6     for i, word in enumerate(words):
7         # Por cada palabra se emite lo que se encontró:
8         # en qué documento y en qué posición
9         yield (word, (key, i))
```

El mapper recorre cada documento palabra por palabra y emite, para cada término encontrado, el documento donde aparece y su posición dentro del texto. Luego el framework agrupa toda esa información por palabra para que el reducer construya el índice invertido.

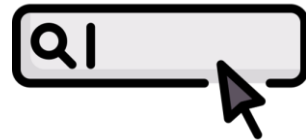
Índices invertidos - MapReduce



```
1 def reducer(key, values):
2     # key: una palabra
3     # values: los documentos y posiciones
4
5     # Sólo queda construir el contenido
6     # del índice para la palabra
7     doc_positions = build_word_content(values)
8
9     yield (key, doc_positions)
```

Este reducer construye una entrada del índice invertido. Recibe una palabra y todas las apariciones de esa palabra en distintos documentos. Luego organiza esa información y genera una estructura que indica en qué documentos aparece la palabra y en qué posiciones, permitiendo realizar búsquedas eficientes.

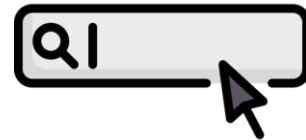
Índices invertidos - Spark



— — —
¿Cuáles eran las características salientes de Spark?

- ❖ Spark gira en torno al concepto de un conjunto de **datos distribuido resiliente (RDD)**, que es una colección de elementos tolerante a fallas que se pueden operar en paralelo.
- ❖ Hay dos formas de crear RDD
 - ❖ **Paralelizar** una colección existente
 - ❖ **Hacer referencia** a un conjunto de datos en un sistema de almacenamiento externo, como un sistema de archivos compartido, HDFS, HBase o cualquier fuente de datos que ofrezca un formato de entrada Hadoop.

Índices invertidos - Spark



```
1 from pyspark.sql import SparkSession
2
3 # Se crea una sesión de Spark
4 spark = SparkSession.builder.appName("InvertedIndexExample").getOrCreate()
5 sc = spark.sparkContext
6
7 documents = [(doc_id, text),...]
8
9 # Se construye un resilient distributed dataset con los documentos
10 docs_rdd = sc.parallelize(documents)
11
12 # Fase de Map: por cada documento se tokeniza y se generan pares (word, (doc, pos))
13 words_rdd = docs_rdd.flatMap(
14     lambda doc: [(word, (doc[0], i)) for i, word in enumerate(doc[1].tokenize())]
15 )
16
17 # Fase de Reduce: se agrupan los elementos de words_rdd por su key (primera columna)
18 # y después se aplica una agregación de lista a los valores (segunda columna)
19 inverted_index_rdd = words_rdd.groupByKey().mapValues(list)
20
21 # Finalmente se traen todos los resultados al proceso central en forma de lista
22 inverted_index = inverted_index_rdd.collect()
```

Filtrado colaborativo

Sistema de recomendación

— — —

Un problema común al que se enfrentan las empresas de Internet es el de recomendar nuevos productos a los usuarios en configuraciones personalizadas (por ejemplo, el sistema de recomendación de productos de Amazon y recomendaciones de películas de Netflix).

- Esto se puede formular como un problema de aprendizaje en el que se nos dan las calificaciones que los usuarios han dado a ciertos elementos y la tarea de **predecir** sus calificaciones **para el resto** de los elementos.
- Formalmente: hay **n** usuarios y **m** elementos, se nos da una **matriz R de $n \times m$** en la cual el j -ésimo valor (u, i) es **r_{ui}** : el rating del elemento i por parte del usuario u .

A la matriz R le faltarán muchas entradas que indican calificaciones no observadas, y nuestra tarea es estimar estas calificaciones no observadas.



Sistema de recomendación

— — —

En nuestra aplicación de streaming de música construida sobre la arquitectura lambda tenemos:

- Eventos sobre **contenido** (música, podcasts, audiolibros)
 - Aparición de nuevo contenido
 - Contenido que es borrado de la plataforma
 - Cambios al contenido (p.ej. agregar o quitar géneros)
- Eventos de interacción de los **usuarios**
 - Reproducir contenido
 - Dar like
 - Compartir
 - Agregar a una playlist

¿Cómo podemos construir una feature de **recomendación** de contenido para aumentar el engagement?



Sistema de recomendación

— — —

Tradicionalmente hay **dos grandes ramas:**

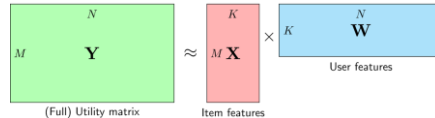
❖ **Content based** filtering

- Recomendarle al usuario cosas *parecidas* a aquello que ya le gustó
- “Porque te gustó X...”

❖ **User based** collaborative filtering

- Recomendarle al usuario cosas que le gustaron a otros usuarios cuyos gustos son parecidos a los suyos

Matrix factorization

$$\mathbf{Y} \approx \hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}$$


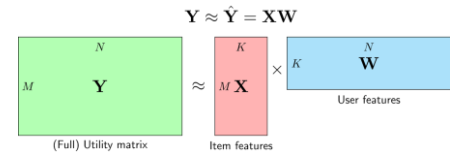
The diagram shows the matrix factorization equation $\mathbf{Y} \approx \hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}$. It includes three matrices: $\mathbf{Y}$ (Full) Utility matrix (green, $M \times N$), $\mathbf{X}$ Item features (red, $M \times K$), and $\mathbf{W}$ User features (blue, $K \times N$). The matrices are arranged to show their dimensions and the relationship between them.

Una técnica para lograr collaborative filtering es la de **matrix factorization**. Para esto primero es necesario construir una matriz de usuario/contenido:

1. Se acumulan **señales (explícitas o implícitas)** de los usuarios respecto del contenido
2. Se define una **función** que toma ese conjunto de señales y devuelve un **puntaje** (positivo o negativo) **normalizado** para cada par [usuario, contenido]
3. Se construye una **matriz (rala)** con los **puntajes**

Matrix factorization

— — —



user/content	song_1	song_2	song_3	song_4	song_5	song_6
user_1	0.6	x	x	-1.2	0.1	-1
user_2	1	0.2	x	x	x	3
user_3	2	0.4	x	x	x	x
user_4	-1.7	x	x	3	x	x

Matrix factorization

$$\mathbf{Y} \approx \hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}$$

The diagram shows three matrices: a green matrix $\mathbf{Y}$ of size $M \times N$ labeled "(Full) Utility matrix", a red matrix $\mathbf{X}$ of size $M \times K$ labeled "Item features", and a blue matrix $\mathbf{W}$ of size $K \times N$ labeled "User features". The equation $\mathbf{Y} \approx \hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}$ is shown above the matrices, with an approximation symbol $\approx$ between $\mathbf{Y}$ and $\hat{\mathbf{Y}}$, and a multiplication symbol $\times$ between $\mathbf{X}$ and $\mathbf{W}$.

- ❖ Esto nos permite conocer cuánto le **gusta o disgusta** un elemento de contenido a un usuario (o el hecho de que no tenemos información al respecto)
- ❖ Eso de por sí ya es útil para tomar decisiones
 - Dejar de mostrar contenido que no le gusta
 - Volver a mostrar contenido de su agrado que hace mucho no visita
 - Encontrar contenido similar al contenido que más le gusta a partir de características conocidas (como banda, género, estilo, etc): **content-based filtering**
- ❖ Pero hay mucho valor en poder determinar qué contenido aún no conocido puede gustarle a partir de **características latentes**

Matrix factorization

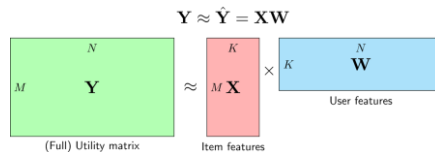
$$\mathbf{Y} \approx \hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}$$

The diagram shows the matrix factorization equation $\mathbf{Y} \approx \hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}$. It includes three matrices: $\mathbf{Y}$ (Full) Utility matrix of size $M \times N$ (green), $\mathbf{X}$ (Item features) of size $M \times K$ (red), and $\mathbf{W}$ (User features) of size $K \times N$ (blue). The approximation symbol $\approx$ is between $\mathbf{Y}$ and $\mathbf{X}\mathbf{W}$, and the multiplication symbol $\times$ is between $\mathbf{X}$ and $\mathbf{W}$.

La técnica de **matrix factorization** permite estimar los puntajes desconocidos a partir de las similitudes entre los puntajes conocidos de distintos usuarios:

user/content	song_1	song_2	song_3	song_4	song_5	song_6
user_1	0.6	0	0	-1.2	0.1	-1
user_2	1	0.2	0	-0.5	0.05	3
user_3	2	0.4	0	-0.8	0.03	2.3
user_4	-1.7	x	0	3	-0.3	1.2

Matrix factorization



— — —

Considerando la **Arquitectura Lambda**:

- ❖ ¿De qué formas podemos **construir los puntajes** para cada usuario/contenido?
- ❖ ¿Cómo pueden **consultarse** para tomar decisiones?
- ❖ ¿Cómo podemos realizar la **matrix factorization**?
- ❖ ¿Qué deberíamos **generar como resultado**, pensando en los posibles casos de uso?

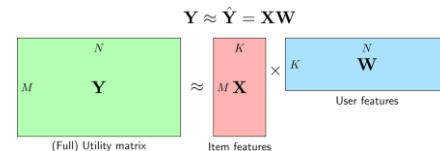
Matrix factorization

$$\mathbf{Y} \approx \hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}$$

The diagram shows the matrix factorization equation $\mathbf{Y} \approx \hat{\mathbf{Y}} = \mathbf{X}\mathbf{W}$. It includes three matrices: $\mathbf{Y}$ (green, $M \times N$, labeled '(Full) Utility matrix'), $\mathbf{X}$ (red, $M \times K$, labeled 'Item features'), and $\mathbf{W}$ (blue, $K \times N$, labeled 'User features'). The matrices are arranged as $\mathbf{Y} \approx \mathbf{X} \times \mathbf{W}$.

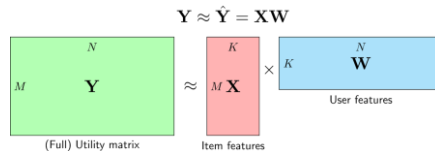
- ❖ Un algoritmo para resolver matrix factorization es el de **alternating least squares (ALS)**, que requiere múltiples iteraciones hasta converger.
- ❖ Pero existen implementaciones **distribuíbles**:
 - Los pasos de ALS implican resolver **ecuaciones lineales**
 - Las ecuaciones pueden resolverse de forma **independiente para cada usuario** o para cada contenido (dependiendo del paso del algoritmo)
 - Las **bibliotecas de Spark** ya cuentan con este tipo de implementaciones

Matrix factorization



```
1 from pyspark.sql import SparkSession
2 from pyspark.ml.evaluation import RegressionEvaluator
3 from pyspark.ml.recommendation import ALS
4 from pyspark.sql import Row
5
6 # Se crea la sesión de Spark
7 spark = SparkSession.builder.appName("MatrixFactorizationExample").getOrCreate()
8
9 data = [Row(userId=x, songId=x, rating=x), ...]
10
11 df = spark.createDataFrame(data)
12
13 # Se construyen datasets de entrenamiento y de prueba
14 (train, test) = df.randomSplit([0.8, 0.2])
15
16 # Se parametriza el model de ALS
17 als = ALS(maxIter=10, regParam=0.01, userCol="userId", itemCol="songId", ratingCol="rating")
18
19 # Se entrena el modelo
20 model = als.fit(train)
21
22 # Se mide su precisión
23 predictions = model.transform(test)
24 evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating", predictionCol="prediction")
25 rmse = evaluator.evaluate(predictions)
26 print(f"Root-mean-square error = {rmse}")
```

Matrix factorization



- ❖ Con el modelo entrenado, se pueden generar recomendaciones o evaluar ratings específicos:

```
1 # Encontrar las 5 mejores canciones para cada usuario
2 userRecs = model.recommendForAllUsers(5)
3
4 # Encontrar los 5 mejores usuarios para cada canción
5 songRecs = model.recommendForAllItems(5)
```

Más detalles:

<https://stanford.edu/~rezab/classes/cme323/S15/notes/lec14.pdf>

Cierre



Bibliografía

 **“Big data: Principles and best practices of scalable realtime data systems”**, N. Marz, J. Warren, 1st edition, 2015.
 Capítulos 4 y 5

 Matrix Completion via Alternating Least Square
<https://stanford.edu/~rezab/classes/cme323/S15/notes/lec14.pdf>